Practical No.: 5

Name: Nikita Lalaso Narute **Roll No.: COB422**

Batch: B2 **Div: B**

Class: BE COMPUTER

Title:

Design an N-Queens matrix having first queen placed. Use backtracking to place remaining queens to generate the final N-Queens matrix.

Program:

```
#include <bits/stdc++.h>
using namespace std;

#define N 4

void printSolution(int board[N][N]) {
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            if (board[i][j])
                cout << "Q ";
            else
                cout << ". ";
        }
        cout << endl;
    }
}

bool isSafe(int board[N][N], int row, int col) {
    int i, j;

    for (i = 0; i < col; i++)
        if (board[row][i])
            return false;

    for (i = row, j = col; i >= 0 && j >= 0; i--, j--)
        if (board[i][j])
            return false;

    for (i = row, j = col; j >= 0 && i < N; i++, j--)
        if (board[i][j])
            return false;

    return true;
}

bool solveNQUtil(int board[N][N], int col) {
    if (col >= N)
        return true;

    for (int i = 0; i < N; i++) {
        if (isSafe(board, i, col)) {
            board[i][col] = 1;

            if (solveNQUtil(board, col + 1))
                return true;

            board[i][col] = 0; // BACKTRACK
        }
    }
    return false;
}

bool solveNQ() {
    int board[N][N] = {0};

    if (!solveNQUtil(board, 0)) {
        cout << "Solution does not exist";
        return false;
    }

    printSolution(board);
    return true;
}

int main() {
    solveNQ();
    return 0;
}
```

Output:

```
(base) sspm@sspm:~$ g++ daa5.cpp
(base) sspm@sspm:~/Desktop$ ./a.out
. . Q .
```

```
Q . . .
. . . Q
. Q . .
(base) sspm@sspm:~$
```